

Python & AI Engineer Interview Preparation

Coding Solutions, Explanations, Complexity, and FastAPI Example

Prepared for Muhammad Shafiullah Khan

How to use this guide: First explain the idea in simple words, then write the code, test it with an example, and finally mention time and space complexity. During an interview, also discuss edge cases.

1. Reverse a String in Python

Explanation

Reversing a string means changing the order of its characters. Python slicing is the simplest approach. The expression `[::-1]` starts from the end and moves backward one character at a time.

Python solution

```
def reverse_string(text: str) -> str:
    return text[::-1]
```

```
word = "Python"
print(reverse_string(word)) # nohtyP
```

Example

Input: "Python" Output: "nohtyP"

Complexity

Time: $O(n)$, because all characters are copied. Space: $O(n)$, because a new string is created.

Interview tip

You can also explain a loop-based method, but slicing is the most Pythonic solution.

2. Check Whether a String Is a Palindrome

Explanation

A palindrome reads the same forward and backward, such as 'madam' or 'level'. We compare the normalized string with its reversed version. Normalization can remove spaces, punctuation, and case differences.

Python solution

```
def is_palindrome(text: str) -> bool:
    cleaned = "".join(char.lower() for char in text if char.isalnum())
    return cleaned == cleaned[::-1]
```

```
print(is_palindrome("Madam")) # True
print(is_palindrome("A man, a plan, a canal: Panama")) # True
```

Example

"Madam" becomes "madam", and its reverse is also "madam", so the result is True.

Complexity

Time: $O(n)$. Space: $O(n)$ because normalized and reversed strings are created.

Interview tip

Ask whether the interviewer wants spaces, punctuation, and letter case to be ignored.

3. Find Duplicate Elements in a List

Explanation

Use one set to store values already seen and another set to store duplicates. A set provides average $O(1)$ lookup, so this is faster than comparing every pair.

Python solution

```
from typing import List

def find_duplicates(numbers: List[int]) -> List[int]:
    seen = set()
    duplicates = set()

    for number in numbers:
        if number in seen:
            duplicates.add(number)
        else:
            seen.add(number)

    return list(duplicates)

print(find_duplicates([1, 2, 3, 2, 4, 1, 5])) # [1, 2]
```

Example

Input: [1, 2, 3, 2, 4, 1, 5] Duplicates: 1 and 2

Complexity

Time: $O(n)$ average. Space: $O(n)$.

Interview tip

Using a duplicate set avoids returning the same duplicate multiple times.

4. Two Sum Problem

Explanation

Given an array and a target, return the indices of two numbers whose sum equals the target. Store each visited number and its index in a dictionary. For every number, calculate the required complement.

Python solution

```
from typing import List

def two_sum(numbers: List[int], target: int) -> List[int]:
    seen = {}

    for index, number in enumerate(numbers):
        complement = target - number
```

```

        if complement in seen:
            return [seen[complement], index]

        seen[number] = index

    return []

```

```
print(two_sum([2, 7, 11, 15], 9)) # [0, 1]
```

Example

At index 1, number 7 needs complement 2. Since 2 was stored at index 0, return [0, 1].

Complexity

Time: $O(n)$ average. Space: $O(n)$.

Interview tip

The brute-force method takes $O(n^2)$. Mention that the dictionary solution is more efficient.

5. Missing Number Problem

Explanation

Assume the array contains distinct numbers from 0 to n with exactly one missing value. The expected sum is $n(n+1)/2$. Subtract the actual sum to obtain the missing number.

Python solution

```

from typing import List

def missing_number(numbers: List[int]) -> int:
    n = len(numbers)
    expected_sum = n * (n + 1) // 2
    actual_sum = sum(numbers)
    return expected_sum - actual_sum

```

```
print(missing_number([3, 0, 1])) # 2
```

Example

For [3, 0, 1], $n = 3$. Expected sum = 6, actual sum = 4, so missing number = 2.

Complexity

Time: $O(n)$. Extra space: $O(1)$.

Interview tip

Clarify the input range. This version assumes values are from 0 through n .

6. Merge Two Sorted Arrays

Explanation

Use two pointers, one for each sorted array. Compare the current values and append the smaller one. After one array is exhausted, append the remaining values from the other array.

Python solution

```

from typing import List

def merge_sorted_arrays(first: List[int], second: List[int]) -> List[int]:
    merged = []
    i = 0
    j = 0

    while i < len(first) and j < len(second):
        if first[i] <= second[j]:
            merged.append(first[i])
            i += 1
        else:
            merged.append(second[j])
            j += 1

    merged.extend(first[i:])
    merged.extend(second[j:])
    return merged

print(merge_sorted_arrays([1, 3, 5], [2, 4, 6]))
# [1, 2, 3, 4, 5, 6]

```

Example

Compare 1 and 2, then 3 and 2, and continue until all values are added in sorted order.

Complexity

Time: $O(n + m)$. Space: $O(n + m)$ for the result.

Interview tip

Do not simply concatenate and sort unless the interviewer allows $O((n+m) \log(n+m))$.

7. Reverse a Linked List

Explanation

A singly linked list node points to the next node. To reverse it iteratively, keep three references: previous, current, and next_node. Change the current node's next pointer to the previous node, then move forward.

Python solution

```

from __future__ import annotations
from typing import Optional

class ListNode:
    def __init__(self, value: int, next_node: Optional[ListNode] = None):
        self.value = value
        self.next = next_node

```

```

def reverse_linked_list(head: Optional[ListNode]) -> Optional[ListNode]:
    previous = None
    current = head

    while current is not None:
        next_node = current.next
        current.next = previous
        previous = current
        current = next_node

    return previous

# 1 -> 2 -> 3 -> None
head = ListNode(1, ListNode(2, ListNode(3)))
reversed_head = reverse_linked_list(head)

while reversed_head:
    print(reversed_head.value, end=" ")
    reversed_head = reversed_head.next

# Output: 3 2 1

```

Example

The pointer direction changes from 1 -> 2 -> 3 to 3 -> 2 -> 1.

Complexity

Time: $O(n)$. Extra space: $O(1)$.

Interview tip

Explain why `next_node` must be saved before changing `current.next`; otherwise, the remaining list is lost.

8. List vs Tuple in Python

Both store ordered collections, but lists are mutable while tuples are immutable. Use a list when values may change and a tuple when the collection should remain fixed.

Feature	List	Tuple
Syntax	[1, 2, 3]	(1, 2, 3)
Mutability	Mutable	Immutable
Methods	More methods	Fewer methods
Performance	Usually slightly slower	Usually slightly faster
Hashable	No	Yes, if all items are hashable
Best use	Changing collection	Fixed record or constant data

```

items = [1, 2, 3]
items[0] = 10      # Allowed

```

```
point = (10, 20)
# point[0] = 5      # TypeError: tuple does not support assignment
```

Interview answer

A list is mutable and uses square brackets. A tuple is immutable and uses parentheses. Tuples are suitable for fixed data and can be dictionary keys when their elements are hashable.

9. Difference Between == and is

The == operator compares values. The is operator checks whether two variables refer to the exact same object in memory.

```
first = [1, 2, 3]
second = [1, 2, 3]
third = first

print(first == second) # True: values are equal
print(first is second) # False: different objects
print(first is third)  # True: same object

value = None
if value is None:
    print("No value")
```

Important rule

Use == for value comparison. Use is mainly for identity checks such as value is None or value is not None.

10. Decorators in Python

A decorator is a function that adds or changes the behavior of another function without modifying that function's original code. Decorators are commonly used for logging, authentication, timing, caching, and validation.

```
from functools import wraps
```

```
def log_call(function):
    @wraps(function)
    def wrapper(*args, **kwargs):
        print(f"Calling {function.__name__}")
        result = function(*args, **kwargs)
        print("Function completed")
        return result

    return wrapper

@log_call
def add(a: int, b: int) -> int:
    return a + b
```

```
print(add(3, 4))
```

How it works

- `@log_call` is equivalent to `add = log_call(add)`.
- wrapper runs extra logic before and after the original function.
- `@wraps` preserves the original function's name and documentation.

11. Generators in Python

A generator produces values one at a time instead of storing the entire result in memory. It uses `yield` rather than `return`. Execution pauses at `yield` and continues from the same position on the next request.

```
from typing import Iterator
```

```
def count_up_to(limit: int) -> Iterator[int]:  
    number = 1
```

```
    while number <= limit:  
        yield number  
        number += 1
```

```
for value in count_up_to(5):  
    print(value)
```

Why generators are useful

- They are memory-efficient for large datasets.
- They support lazy evaluation.
- They are useful for file processing, data streams, and pipelines.

Complexity

Each value is generated in $O(1)$ time, and the generator uses $O(1)$ extra memory.

12. `*args` and `**kwargs`

`*args` collects extra positional arguments into a tuple. `**kwargs` collects extra keyword arguments into a dictionary. They are useful when a function needs to accept a flexible number of arguments.

```
def show_details(name: str, *skills: str, **profile) -> None:  
    print("Name:", name)  
    print("Skills:", skills)  
    print("Profile:", profile)
```

```
show_details(  
    "Shafiullah",  
    "Python",  
    "LangChain",  
    role="AI Engineer",  
    experience="Trainee"
```

)

Result

skills becomes ('Python', 'LangChain'), while profile becomes {'role': 'AI Engineer', 'experience': 'Trainee'}.

Interview tip

The names args and kwargs are conventional; the * and ** symbols create the behavior.

13. Multithreading vs Multiprocessing

Multithreading runs multiple threads inside one process. Multiprocessing runs multiple independent processes. The best choice depends on whether the task waits for I/O or performs heavy CPU computation.

Feature	Multithreading	Multiprocessing
Execution unit	Threads	Processes
Memory	Shared	Separate
Best for	I/O-bound tasks	CPU-bound tasks
Examples	API calls, downloads, files	Image processing, calculations
Overhead	Lower	Higher
Python GIL	Limits CPU parallelism	Each process has its own interpreter

```
# Threading example: suitable for I/O-bound work
from concurrent.futures import ThreadPoolExecutor

def download_file(file_name: str) -> str:
    return f"Downloaded {file_name}"

with ThreadPoolExecutor(max_workers=3) as executor:
    results = executor.map(download_file, ["a.pdf", "b.pdf", "c.pdf"])

print(list(results))
```

Interview answer

Use multithreading for I/O-bound tasks because threads can work while another thread is waiting. Use multiprocessing for CPU-bound work because separate processes can run on multiple CPU cores.

14. Build a REST API with FastAPI

A REST API exposes resources through HTTP endpoints. FastAPI uses Python type hints and Pydantic models for automatic validation, and it automatically generates Swagger documentation.

Install dependencies

```
pip install fastapi uvicorn
```

main.py

```
from typing import Dict, List

from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field
```

```

app = FastAPI(
    title="Interview Practice API",
    version="1.0.0"
)

class CandidateCreate(BaseModel):
    name: str = Field(min_length=2, max_length=100)
    skills: List[str] = Field(default_factory=list)
    experience_years: float = Field(ge=0)

class Candidate(CandidateCreate):
    id: int

candidates: Dict[int, Candidate] = {}
next_id = 1

@app.get("/")
def home() -> dict:
    return {"message": "AI Engineer Interview API is running"}

@app.get("/candidates", response_model=List[Candidate])
def get_candidates() -> List[Candidate]:
    return list(candidates.values())

@app.get("/candidates/{candidate_id}", response_model=Candidate)
def get_candidate(candidate_id: int) -> Candidate:
    candidate = candidates.get(candidate_id)

    if candidate is None:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Candidate not found"
        )

    return candidate

```

```

@app.post(
    "/candidates",
    response_model=Candidate,
    status_code=status.HTTP_201_CREATED
)
def create_candidate(data: CandidateCreate) -> Candidate:
    global next_id

    candidate = Candidate(id=next_id, **data.model_dump())
    candidates[next_id] = candidate
    next_id += 1
    return candidate

@app.delete(
    "/candidates/{candidate_id}",
    status_code=status.HTTP_204_NO_CONTENT
)
def delete_candidate(candidate_id: int) -> None:
    if candidate_id not in candidates:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="Candidate not found"
        )

    del candidates[candidate_id]

```

Run the API

```
uvicorn main:app --reload
```

Open documentation

After starting the server, open <http://127.0.0.1:8000/docs> for interactive Swagger documentation.

Example POST request

POST /candidates

```

{
  "name": "Muhammad Shafiullah Khan",
  "skills": ["Python", "LangChain", "RAG"],
  "experience_years": 1
}

```

Important FastAPI concepts

- GET retrieves data.
- POST creates data.
- DELETE removes data.
- Pydantic validates request and response data.
- HTTPException returns proper errors such as 404 Not Found.
- response_model documents and validates the response format.
- For production, replace the in-memory dictionary with a real database.

Final Interview Checklist

- Explain the approach before writing code.
- Confirm assumptions and input constraints.
- Discuss edge cases such as empty input and duplicate values.
- State time and space complexity.
- Write readable variable names and type hints.
- Test the solution with at least one example.
- For AI roles, connect Python concepts to APIs, RAG pipelines, and agent workflows.